ZentroMart
Support that listens.

Shivam Maurya | Implementation and measured evaluation

Problem and approach

Build one conversational customer-support agent for the supplied fictional store. The agent must answer from company documents, verify identity before accessing an order, remember follow-ups, create genuine mock handover tickets, and listen and speak through Gnani.

Step 1 - Prompt-only baseline

A hosted Gemini model receives the support-assistant role and customer question without company documents, retrieval results or order tools. The same practice questions are used for the grounded version. Baseline replies are saved exactly as generated; no test answers are written or edited manually.

Step 2 - Grounded document search

The loader preserves section IDs, document type, effective date and current/superseded status. It creates 768-dimensional Gemini Embedding 2 vectors and stores a fingerprinted local vector index. Cosine similarity and BM25 rankings are combined using reciprocal-rank fusion. Exact SKU lookup and limits on similar catalog rows prevent products from crowding out policy evidence. Relevant policy documents are expanded to include their exceptions. Public pincode data is looked up directly.

The index contains 154 document sections and product rows. A catalog-wide category summary is derived from the product CSV at retrieval time for broad store questions. Current policies take precedence over agent guides, help, FAQs and marketing. Superseded returns rules apply only to the historical exception allowed by the current policy. Historical support logs are not policy authority.

Step 3 - Conversation and tools

Each conversation has isolated history and verification state. Python calls the supplied lookup_order tool, enforces one retry, and withholds private order details until verification succeeds. Verification secrets are redacted from stored traces. Handover claims require a successful supplied create_ticket call. Dates are computed against 20 October 2026; business-day calculations exclude Sundays and the supplied holidays, while return windows use calendar days.

Measured practice evaluation

These are local heuristic measurements, not official competition scores. API failures remain visible and incomplete runs are not described as complete.

Measure	Prompt only	Grounded RAG
Questions attempted	52/52	52/52
Expected decision	18/52 (34.6%)	48/52 (92.3%)
Required fact-string recall	11/58 (19.0%)	55/58 (94.8%)
Gold evidence recall	No retrieval	90/90 (100.0%)
API errors	0	0

Multi-turn checks

Conversation turns attempted: 96/96 across the supplied 23 conversations. Expected-decision matches: 91/96 (94.8%). Tool-name sequence matches: 95/96 (99.0%). API errors: 0.

Conversation fact-string recall: 71/82 (86.6%); gold evidence recall: 109/112 (97.3%).

What the measurements mean

The evaluator compares decisions, required fact strings, forbidden phrases and gold source IDs. Numeric matching avoids counting a warranty duration merely because the same digits occur inside a SKU. Tool checks compare names; they do not establish full argument correctness. Equivalent wording, translated dates and negated statements need manual review. Retrieval recall does not measure source precision. Full raw generated replies and evidence are supplied as JSON for inspection.

Issues found and improvements

Initial SKU searches over-retrieved near-identical product rows and missed the warranty claim procedure. Balanced retrieval corrected that failure. The agent also requested unnecessary order verification for a customer-supplied refund date; deterministic scenario arithmetic and clearer instructions address this. Conversation testing found an escalation triggered by background damage immediately after verification; verification-only turns now acknowledge success without inventing an active issue.

Return-deadline testing exposed confusion between placement and delivery dates for festive eligibility; the agent now receives explicit item-specific deadlines. A separate post-fix recheck of seven weak single-question cases matched six decisions, with all required fact strings present. Original full-run results above remain unchanged. The remaining case refused private data correctly in prose but returned the wrong decision label.

Manual review also found missing pickup-fee and refund details, an unnecessary escalation, and refusal-label inconsistencies in conversations. Some fact-string misses were equivalent Hindi wording or date formatting. All 25 local automated checks passed, including from a clean public GitHub download; these checks do not prove every generated answer is correct.

Model identifiers observed in saved outputs: gemini-3.5-flash-lite.

Voice and reproducibility

Step 4 - Gnani speech integration

The local interface records short microphone turns as 16 kHz mono PCM WAV. Gnani STT converts the recording to text, the same grounded conversational agent responds, and Gnani Timbre v2.5 produces reply audio. No other speech recognition or speech synthesis service is used. A live microphone test produced matching written and spoken answers; the user observed approximately ten seconds for one turn, which is an informal observation rather than a latency benchmark.

A speech round-trip check confirmed synthesis and transcription. An initial Gnani gateway 1010 rejection was resolved by sending an explicit application User-Agent. Intermittent STT 403 errors were later observed even with that header; subsequent health checks succeeded, but their cause remains unconfirmed. Gemini quota errors and service unavailability were also observed. Failures are shown explicitly; bounded retries apply to transient status codes, not authorization failures.

Install and batch run

Python 3.10+ is required. Runtime integrations use the standard library and official REST APIs. Obtain the original dataset separately, provide Gemini and Gnani keys through environment variables or hidden prompts, and run:

```
python run.py --dataset "/path/to/dataset"
--input "test_audio" --output "sample_outputs/batch"
```

Root audio files are independent unless --conversation is specified. Subfolders represent conversations with zero-padded filenames for turn order. Outputs include text, Gnani audio, retrieved chunks, citations, sanitized tools and a manifest. Failed inputs are recorded, later inputs continue, and the batch exits nonzero on any failure.

A live three-turn batch through the downloaded public run.py passed: Wallet refund, card follow-up and actual mock ticket creation, with three spoken replies in 30.66 seconds total. Input clips were synthesized using Gnani. Existing connected API client objects and the saved index were reused; no API service was mocked. Typed key prompts and a cold index rebuild were not retested. See sample_outputs/batch_live/check_summary.json.

Privacy and limitations

Only the supplied synthetic customer data is used. The local demo stores keys in process memory and never reads private key files. Runtime caches, tickets, .env, API keys, dataset copies and virtual environments are excluded from packaging. The supplied phone-suffix/pincode check is an educational simulation, not production authentication. The local demo is for one user and is not intended for internet hosting.

Remaining improvement areas include broader native-language review, policy-faithfulness validation of generated dates and conditions, stronger prompt-injection testing, quantitative latency measurements and resilience to provider outages. No production phone-call deployment or bonus feature is claimed.

Submission artifacts and references

Public source: github.com/shivammaurya0017/zentromart-voiceops. Installation: README.md. Evaluation: sample_outputs/. The genuine human-spoken demo is 113.5 seconds, includes follow-up, document evidence and handover, and is supplied as demo.mp4. This report is three pages, within the four-page limit.

References: VoiceOps 2026 Capstone Brief; supplied README.md and DATA_DICTIONARY.md; docs.gnani.ai/api/introduction/quick-start; ai.google.dev/api/generate-content; ai.google.dev/gemini-api/docs/embeddings.